INF1103: Lab 4

Data Persistence

1. Introduction

Welcome to Week 4! Over the last three weeks, you have built a modular "Auditor" that runs in memory.

Remember, the focus of this module is all about bridging logical programming constructs with modern, automated operational practices. We will systematically evolve our applications week by week, refactoring and packaging our code to fit enterprise-grade deployment standards.

The expectation is that you complete and verify your lab by the end of your scheduled lab session. True engineering mastery comes from hands-on problem-solving. Ensure you work through and complete each step of this lab by yourself. While your peers are excellent resources for discussing conceptual architectures, they must not write code for you or share their files with you!

It is highly critical that you have watched and engaged with the Week 1 lecture lessons before attempting this lab. We assume you have acquired the foundational mental models—like standard shipping containers, blueprints vs. instances, and environment drift—which you will now put into practice.

Do not hesitate to ask plenty of questions to your professor and teaching assistants when you hit an infrastructure error. We are here to support you as you take your first steps into the DevOps pipeline. Good luck!

We have encountered a fundamental limitation: **when the container stops, your data disappears.** This week, we bridge that gap. You will learn how to make your data survive application restarts using **File Handling** and how to manage multiple data points using **Arrays (Lists)**.

Remember:

- Data persistence is the difference between a "script" and an "application."
- Focus on how we read data into memory at startup and write data back to disk at shutdown.

2. Assignment

1. The Scenario

The store manager needs a system that remembers inventory levels even after the program closes. Furthermore, they need to store a history of all transaction amounts, not just the running total.

Requirements

Copy your modular_auditor.py to persistent_auditor.py.

1. **Persistence:** At the start of the program, read the information previously saved in the inventory file. If the inventory file does not exist, start with an empty inventory and continue running without producing an error.
2. **History Tracking:** Use a Python list (array) to store every valid transaction amount entered.
3. **Write-Back:** When the user types quit, save the final total and the transaction history list to inventory.txt.
4. **Modularity:** Maintain your functional design. Create a load_inventory() and save_inventory() function.

   Sample expected output:

```
Current Orders:

1001, Wireless Mouse, 2
1002, Keyboard, 1
1003, USB Cable, 3

Enter Product Name: Laptop Stand
Enter Quantity: 2

New Order Added:
1004,Laptop Stand,2

Order successfully saved to orders.txt
```

2. Initialize Your Workspace & Git

1. Initialize and track your work.
2. Commit your progress after you successfully complete each phase:
   a. After load_inventory() is working.
   b. After the list is successfully tracking history.
   c. After the final save_inventory() is verified.
3. Finally push the code to your github repository.

3. Dockerize with Volumes

To ensure your file persists outside the container even if the container is destroyed, run it with a volume mount.

What you need to submit?

1. Your python file (persistent_auditor.py)
2. Your updated inventory.txt text file
3. Your git log. Store it in a text file.
   git log >log.txt
4. Docker Mount and run. A screenshot of the successful execution is usually sufficient.
5. Run the following command for evidence that the image was successfully built. Screenshot of the output.

   ```
   docker images
   ```

6. Proof that the image runs successfully with a screenshot showing the program accepting input and producing results.

7. Put everything in a folder. Zip it and upload to xSite.